27

Energy-Dependent Parameters

This chapter describes extensions for the temperature-dependent models.

- Overview
- Energy-Dependent Energy Relaxation Time
- Energy-Dependent Mobility
- Energy-Dependent Peltier Coefficient

Overview

Sentaurus Device provides the possibility to specify some parameters as a ratio of two irrational polynomials. The general form of such ratio is written as:

Equation 946

$$G(w,s) = f\frac{\left(\left(\sum a_i w^{p_i}\right) + d_n s\right)^{g_n}}{\left(\left(\sum a_j w^{p_j}\right) + d_d s\right)^{g_d}}$$

where:

- Subscripts n and d correspond to the numerator and denominator, respectively.
- f is a factor, w is a primary variable, and s is an additional variable.

You can use Equation 946 with different coefficients for different intervals k defined by the segment $\left[w_{k-1}^{\max}, w_k^{\max}\right]$. By default, it is assumed that only one interval $k = 0$ with the boundaries $[0, \infty]$ exists, and function G is constant, that is, $a_0 = 0$, $a_i = 0$, $p = d = 0$, $g = 1$. Factor f is defined accordingly for each model.

A simplified syntax is introduced to define the piecewise linear function G. The boundaries of the intervals and the value of factor must be specified, which means the value of G is at the right side of the interval. All other coefficients should not be specified to use this possibility. As there are some peculiarities in parameter specification and model activation, the specific models for which the approximation by Equation 946 is supported are described here separately.

Energy-Dependent Energy Relaxation Time

For the specification of the energy relaxation time, the following modification of Equation 946 is used:

Equation 947

$$\tau(w) = \tau_w^0 \frac{\left(\left(\sum a_i w^{p_i}\right)\right)^{g_n}}{\left(\left(\sum a_j w^{p_j}\right)\right)^{g_d}}$$

where $w = 1.5kT_n / q$ for electrons and $w = kT_p / q$ for holes.

The factor f in Equation 946 is defined by τ_w^0, which can be specified in the parameter file by the values `tau_w_ele` and `tau_w_hol`.

To activate the specification of the energy-dependent energy relaxation time, the parameter `Formula(tau_w_ele)` (or `Formula(tau_w_hol)` for holes) must be set to 2.

The following example shows the energy relaxation time section of the parameter file and provides a short description of the syntax:

```
EnergyRelaxationTime
{ *  Energy relaxation times in picoseconds
      tau_w_ele = 0.3   # [ps]
      tau_w_hol = 0.25  # [ps]
 * Below is the example of energy relaxation time approximation
 * by the ratio of two irrational polynomials.
 * If Wmax(interval-1) < Wc < Wmax(interval), then:
 * tau_w = (tau_w)*(Numerator^Gn)/(Denominator^Gd),
 * where (Numerator or Denominator)=SIGMA[A(i)(Wc^P(i))],
 * Wc=1.5(k*Tcar)/q (in eV).
 * By default: Wmin(0)=Wmax(-1)=0; Wmax(0)=infinity.
 * The option can be activated by specifying appropriate Formula
 * equals 2
 *      Formula(tau_w_ele) = 2
 *      Formula(tau_w_hol) = 2
 *      Wmax(interval)_ele =
 *      tau_w_ele(interval) =
 *      Numerator(interval)_ele{
 *        A(0) =
 *        P(0) =
 *        A(1) =
 *        P(1) =
 *        D    =
 *        G    =
 *      }
 *      Denominator(interval)_ele{
 *        A(0) =
 *        P(0) =
 *        D    =
```

```
*        G    =
*      }
*      Wmax(interval)_hol =
*      tau_w_hol(interval) =
  tau_w_ele = 0.3   # [ps]
  tau_w_hol = 0.25  # [ps]

  Formula(tau_w_ele) = 2
  Numerator(0)_ele{
     A(0) = 0.048200
     P(0) = 0.00
     A(1) = 1.00
     P(1) = 3.500
     A(2) = 0.0500
     P(2) = 2.500
     A(3) = 0.0018100
     P(3) = 1.00
  }
  Denominator(0)_ele{
     A(0) = 0.048200
     P(0) = 0.00
     A(1) = 1.00
     P(1) = 3.500
     A(2) = 0.100
     P(2) = 2.500
  }
```

The following example shows a simplified syntax for piecewise linear specification of energy relaxation time:

```
EnergyRelaxationTime:
{ *  Energy relaxation times in picoseconds
      Formula(tau_w_ele) = 2
      tau_w_ele = 0.3       # [ps]

      Wmax(0)_ele = 0.5     # [eV]
      tau_w_ele(1) = 0.46   # [ps]

      Wmax(1)_ele = 1.0     # [eV]
      tau_w_ele(2) = 0.4    # [ps]

      Wmax(2)_ele = 2.0     # [eV]
      tau_w_ele(3) = 0.2    # [ps]

      tau_w_hol = 0.25      # [ps]
}
```

Spline Interpolation

Sentaurus Device also allows spline approximation of energy relaxation time over energy. In this case, the parameter `Formula(tau_w_ele)` for electron energy relaxation time (and

similarly, parameter `Formula(tau_w_hol)` for hole energy relaxation time) must be equal to 3.

Inside the braces following the keyword `Spline(tau_w_ele)` (or `Spline(tau_w_hol)`), an energy [eV] and tau [ps] value pair must be specified in each line. For the values outside of the specified intervals, energy relaxation time is treated as a constant and equal to the closest boundary value.

The following example shows a spline approximation specification for energy-dependent energy relaxation time for electrons:

```
EnergyRelaxationTime {
   Formula(tau_w_ele) = 3
      Spline(tau_w_ele) {
         0.     0.3   # [eV] [ps]
         0.5    0.46  # [eV] [ps]
         1.     0.4   # [eV] [ps]
         2.     0.2   # [eV] [ps]
      }
}
```

Note:

Energy relaxation times can be either energy-dependent or mole fraction–dependent (see Abrupt and Graded Heterojunctions on page 62), but not both.

Energy-Dependent Mobility

In addition to existing energy-dependent mobility models (such as Caughey–Thomas, where the effective field is computed inside Sentaurus Device as a function of the carrier temperature), a more complex, user-supplied mobility model can be defined. For such a specification of energy-dependent mobility, a modification to Equation 946 is used:

Equation 948

$$\mu\left(\bar{w}, N_{\mathrm{tot}}\right) = \mu_{\mathrm{low}} \frac{\left(\left(\sum a_i \bar{w}^{p_i}\right) + d_n N_{\mathrm{tot}}\right)^{g_n}}{\left(\left(\sum a_j \bar{w}^{p_j}\right) + d_d N_{\mathrm{tot}}\right)^{g_d}}$$

where $\bar{w} = T_n / T$ for electrons or $\bar{w} = T_p / T$ for holes, and μ_{low} is the low field mobility.

To activate the model, you must specify `CarrierTempDrivePolynomial`, the driving force keyword, as a parameter of the high-field saturation mobility model. Parameters of the polynomials must be defined in the `HydroHighFieldMobility` parameter set.

This example shows the output of the `HydroHighFieldMobility` section and the specification of coefficients:

```
HydroHighFieldDependence:
  { * Parameter specifications for the high field degradation in
    * some hydrodynamic models.
    * B) Approximation by the ratio of two irrational polynomials
    * (driving force 'CarrierTempDrivePolynomial'):
    * If Wmax(interval-1) < w < Wmax(interval), then:
    * mu_hf = mu*factor*(Numerator^Gn)/(Denominator^Gd),
    * where (Numerator or Denominator)={SIGMA[A(i)(w^P(i))]+D*Ni},
    * w=Tc/Tl; Ni(cm^-3) is total doping.
    * By default: Wmin(0)=Wmax(-1)=0; Wmax(0)=infinity.

    *      Wmax(interval)_ele =
    *      F(interval)_ele =
    *      Numerator(interval)_ele{
    *        A(0)  =
    *        P(0)  =
    *        A(1)  =
    *        P(1)  =
    *        D     =
    *        G     =
    *      }
    *      Denominator(interval)_ele{
    *        A(0)  =
    *        P(0)  =
    *        D     =
    *        G     =
    *      }
    *      F(interval)_hol =
    *      Wmax(interval)_hol =
         Denominator(0)_ele
  {
          A(0)  = 0.3
          P(0)  = 0.0
          A(1)  = 1.0
          P(1)  = 2.
          A(2)  = 0.001
          P(2)  = 2.500
          D     = 3.00e-16
          G     = 0.2500
         }
}
```

Spline Interpolation

Instead of the rational polynomial in Equation 948, you can use a spline interpolation. In this case, the option `CarrierTempDriveSpline` must be used for the high-field mobility model in the command file.

For example:

```
Physics {
   Mobility (
      HighFieldSaturation (CarrierTempDriveSpline)
   )
}
```

The energy-dependent mobility is computed as:

Equation 949

$$\mu(\bar{w}) = \mu_{\mathrm{low}} \cdot \mathrm{spline}(\bar{w})$$

where the function spline(*w*) is defined by a sequence of value pairs in the parameter file:

```
HydroHighFieldDependence {
   Spline (electron) {
      0   1
      1   1
      2   2.5
      4   4
      10  5
   }

   Spline (hole) {
      0   1
      1   1
      2   0.75
      4   0.5
      10  0.2
   }
}
```

The given data points are interpolated by a cubic spline. Zero derivatives are imposed as boundary conditions at the end points. The spline function remains constant beyond the end points.

Energy-Dependent Peltier Coefficient

Sentaurus Device allows for the following modification of the expression of the energy flux equation:

Equation 950

$$\vec{S}_n = -\frac{5r_n}{2}\left(\frac{kT_n}{q}\vec{J}_n + f_n^{\mathrm{hf}}\hat{\kappa}_n\frac{\partial(w_n\Pi_n)}{\partial w_n}\nabla T_n\right)$$

The standard expression corresponds to $\Pi_n = 1$. If $\Pi_n = 1 + P(\bar{w})$, then:

Equation 951

$$\frac{\partial(w_n \Pi_n)}{\partial w_n} = 1 + \bar{w}\frac{\partial P(\bar{w})C}{\partial \bar{w}}$$

Sentaurus Device allows you to specify the function Q:

Equation 952

$$Q(\bar{w}) = \bar{w}\frac{\partial P(\bar{w})}{\partial \bar{w}}$$

For the specification of Q, the following modification of Equation 946 is used:

Equation 953

$$Q(\bar{w}) = f\frac{\left(\left(\sum a_i \bar{w}^{p_i}\right)\right)^{g_n}}{\left(\left(\sum a_j \bar{w}^{p_j}\right)\right)^{g_d}}$$

Coefficients must be specified in the `HeatFlux` parameter set, and the dependence can be activated by specifying a nonzero factor f.

For $\Pi_n = 1 + 1/\left(\sqrt{1+\bar{w}^2}\right)$, the result is $Q = 1/\left(\bar{w}^2+1\right)^{1.5}$.

This is an example of the parameter file section for such a function Q_n:

```
HeatFlux
{ *  Heat flux factor (0 <= hf <= 1)
      hf_n = 1     # [1]
      hf_p = 1     # [1]
 * Coefficients can be defined also as:
 *      hf_new = hf*(1.+Delta(w))
 * where Delta(w) is the ratio of two irrational polynomials.
 * If Wmax(interval-1) < Wc < Wmax(interval), then:
 * Delta(w) = factor*(Numerator^Gn)/(Denominator^Gd),
 * where (Numerator or Denominator)=SIGMA[A(i)(w^P(i))], w=Tc/Tl
 * By default: Wmin(0)=Wmax(-1)=0; Wmax(0)=infinity.
 * Option can be activated by specifying nonzero 'factor'.
 *      Wmax(interval)_ele =
 *      F(interval)_ele = 1
 *      Numerator(interval)_ele{
 *        A(0)  =
 *        P(0)  =
 *        A(1)  =
 *        P(1)  =
 *        G     =
 *      }
 *      Denominator(interval)_ele{
 *        A(0)  =
```

```
*         P(0)   =
*         G      =
*       }
*       Wmax(interval)_hol =
*       F(interval)_hol = 1
        f(0)_ele = 1
      Denominator(0)_ele{
        A(0)   = 1.
        P(0)   = 0.
        A(1)   = 1.
        P(1)   = 2.
        G      = 1.5
      }
```

Spline Interpolation

Instead of the rational polynomial in Equation 953, a spline interpolation can be used as well. The function $Q(\bar{w})$ is defined in the parameter file by a sequence of value pairs:

```
HeatFlux {
   hf_n = 1
   hf_p = 1

   Spline (electron) {
      0   1
      1   1
      2   2.5
      4   4
      10  5
   }

   Spline (hole) {
      0   1
      1   1
      2   0.75
      4   0.5
      10  0.2
   }
}
```

The given data points are interpolated by a cubic spline. Zero derivatives are imposed as boundary conditions at the end points. The spline function remains constant beyond the end points.